DATA STRUCTURES & ALGORITHMS · C++

Stacks & Queues Solved Questions

Every problem from Part 11 of the Stacks & Queues guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38	3	12	143
PROBLEMS	TIERS	PATTERNS	TESTS PASSING

Prepared for **Hari** · on-campus placement preparation
Companion to **Stacks-and-Queues-Guide.pdf** · code verified against **reference-part11-solutions.cpp**
Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.
Generated 08 September 2026

Contents

32 LC 456 · 132 Pattern

33 LC 316 · Remove Duplicate Letters

34 LC 1019 · Next Greater Node In Linked List

35 LC 636 · Exclusive Time of Functions

36 LC 895 · Maximum Frequency Stack

37 LC 862 · Shortest Subarray with Sum at Least K

38 LC 1425 · Constrained Subsequence Sum

Tier 1 — Learn the mechanics

Thirteen problems. When you can write all thirteen from a blank page, you have the mechanics of both containers. Do not move on before that is true.

01 LC 20 · Valid Parentheses

EASY Pattern 1 — Bracket matching

PROBLEM

Given a string containing only `(`, `)`, `{`, `}`, `[` and `]`, decide whether it is valid: every bracket is closed by the same type, and closed in the correct order.

APPROACH

Scan left to right. Push every opening bracket; on a closing bracket the top of the stack must be its partner. A hash map from closer to opener keeps the code short and scales to more bracket types.

The stack is holding exactly what Part 2.1 says it holds — the openers that are **still unresolved**. When the matching closer arrives, that opener is resolved and comes off.

Complexity O(n) time · O(n) space in the worst case, when the whole string is openers

C++ SOLUTION

```cpp
class Solution {
public:
    bool isValid(string s) {
        unordered_map<char, char> match{{')', '('}, {']', '['}, {'}', '{'}};
        stack<char> st;
        for (char c : s) {
            if (!match.count(c)) {              // an opening bracket
                st.push(c);
            } else {                            // a closing bracket
                if (st.empty() || st.top() != match[c]) return false;
                st.pop();
            }
        }
        return st.empty();                      // nothing may be left unclosed
    }
};
```

WATCH OUT

Three checks are needed and dropping any one is a wrong answer: `st.empty()` **before** popping (catches `")("`), the partner comparison (catches `"([)]"`), and `st.empty()` at the **end** (catches `"(("`). The counter shortcut — increment on `(`, decrement on `)` — works only when there is a single bracket type; with three types it happily accepts `"([)]"`.

02 LC 921 · Minimum Add to Make Parentheses Valid

MEDIUM Pattern 1 — the counter shortcut

PROBLEM

Given a string of `(` and `)`, return the minimum number of brackets you must insert anywhere to make it valid.

APPROACH

Only one bracket type, so no stack is needed — two counters do it. `open` tracks unmatched `(` seen so far; `need` counts `)` that arrived with nothing to match. A `)` either cancels an open bracket or becomes an orphan.

The answer is `open + need`: every leftover opener needs a closer, and every orphan closer needs an opener.

Complexity O(n) time · **O(1) space**

C++ SOLUTION

```cpp
class Solution {
public:
    int minAddToMakeValid(string s) {
        int open = 0;        // unmatched '(' seen so far
        int need = 0;        // ')' that had nothing to match
        for (char c : s) {
            if (c == '(') open++;
            else if (open > 0) open--;          // this ')' matches an earlier '('
            else need++;                        // an orphan ')'
        }
        return open + need;
    }
};
```

WATCH OUT

Do not reach for a stack out of habit — with one bracket type the counter is strictly better and the interviewer is checking whether you notice. But be able to say **why** it only works here: a counter cannot distinguish bracket *types*, which is exactly what LC 20 needs.

03 LC 155 · Min Stack

MEDIUM Pattern 11 — Design with an auxiliary structure

PROBLEM

Design a stack supporting `push`, `pop`, `top` and `getMin`, **all in O(1)**.

APPROACH

Keep a second, *shadow* stack whose top is always the minimum of everything currently in the main stack. On every push, push `min(newValue, mins.top())`. On every pop, pop both.

Push onto `mins` **unconditionally**, even when the new value is not a new minimum. That keeps the two stacks exactly the same height, so `pop()` is a single line and the two can never desynchronise.

Complexity O(1) for every operation · O(n) space

C++ SOLUTION

```cpp
class MinStack {
    stack<int> st;       // the real stack
    stack<int> mins;     // mins.top() == minimum of everything currently in st
public:
    MinStack() {}

    void push(int val) {
        st.push(val);
        mins.push(mins.empty() ? val : min(val, mins.top()));  // push ALWAYS
    }
    void pop()    { st.pop(); mins.pop(); }     // the two stay in lockstep
    int  top()    { return st.top(); }
    int  getMin() { return mins.top(); }
};
```

WATCH OUT

The 'optimisation' of pushing onto `mins` only when `val <= mins.top()` also works, but then `pop` must compare before popping `mins` — and if you write `<` instead of `<=` there, duplicate minima break it. The O(1)-extra-space follow-up stores the encoded value `2*val - min` and decodes on pop; mention it, but say the two-stack version is what you would ship.

04 LC 232 · Implement Queue using Stacks

EASY Pattern 11 — Design, amortised O(1)

PROBLEM

Implement a FIFO queue using only two LIFO stacks, supporting `push`, `pop`, `peek` and `empty`.

APPROACH

`in` receives every push. `out` serves every pop. Moving everything from `in` to `out` reverses the order, which is exactly the LIFO-to-FIFO conversion.

The critical design decision is **lazy refill**: only shift when `out` is empty, and when you do, shift *everything*. That is what makes the amortised bound hold.

> **Complexity** `push` O(1) · `pop` and `peek` **amortised O(1)**, O(n) worst case · O(n) space

C++ SOLUTION

```cpp
class MyQueue {
    stack<int> in;       // everything pushed goes here
    stack<int> out;      // everything popped comes from here, in reverse order

    void shift() {                              // only ever called when out is empty
        while (!in.empty()) { out.push(in.top()); in.pop(); }
    }
public:
    MyQueue() {}

    void push(int x) { in.push(x); }

    int pop() {
        peek();                                 // makes sure out is non-empty
        int v = out.top(); out.pop();
        return v;
    }
    int peek() {
        if (out.empty()) shift();               // LAZY refill — the whole point
        return out.top();
    }
    bool empty() { return in.empty() && out.empty(); }
};
```

> **WATCH OUT**
>
> Say the amortised argument out loud, because it is the whole point of the question: *every element moves from `in` to `out` exactly once in its lifetime, so n operations cost O(n) in total.* Refilling `out` on every pop still gives correct answers but is genuinely O(n) per operation — and it is the wrong answer to the complexity follow-up.

05 LC 225 · Implement Stack using Queues

EASY Pattern 11 — Design, rotate on push

PROBLEM

Implement a LIFO stack using only FIFO queues, supporting `push`, `pop`, `top` and `empty`.

APPROACH

This direction cannot be made amortised O(1) — one of the two operations has to be O(n), and you choose which. The tidiest is a **single queue with a rotation on push**: push the new element, then move every *previous* element from the front to the back, so the newest element ends up at the front where `pop` can reach it.

One container, O(n) push, O(1) pop.

> **Complexity** `push` O(n) · `pop`, `top`, `empty` O(1) · O(n) space

C++ SOLUTION

```cpp
class MyStack {
    queue<int> q;
public:
    MyStack() {}

    void push(int x) {
        q.push(x);
        // rotate every OTHER element to the back, so x ends up at the front
        for (int i = 1; i < (int)q.size(); i++) { q.push(q.front()); q.pop(); }
    }
    int  pop()   { int v = q.front(); q.pop(); return v; }
    int  top()   { return q.front(); }
    bool empty() { return q.empty(); }
};
```

> **WATCH OUT**
>
> The loop bound is `i < q.size()` starting at `i = 1`, not `0` — you rotate the *other* `size - 1` elements, not all of them. Rotating all of them puts the new element back at the rear and the stack behaves like a queue. The two-queue variant with an O(n) pop is equally valid; say which operation you optimised for and why.

06 LC 622 · Design Circular Queue

MEDIUM Pattern 11 — Design, the wrap-around

PROBLEM

Implement a fixed-capacity FIFO queue on an array, reusing the space freed by dequeues. All operations O(1).

APPROACH

Store `front` and `count` and **derive** the rear as `(front + count) % cap`. Enqueue writes at that derived slot and increments `count`; dequeue just walks `front = (front + 1) % cap` forward and decrements.

The modulus is what turns the array into a ring, so the free space behind `front` is reused instead of being lost.

> **Complexity** O(1) for every operation · O(k) space

C++ SOLUTION

```cpp
class MyCircularQueue {
    vector<int> data;
    int cap, front_ = 0, count = 0;      // rear is DERIVED, never stored
public:
    MyCircularQueue(int k) : data(k), cap(k) {}

    bool enQueue(int value) {
        if (isFull()) return false;
        data[(front_ + count) % cap] = value;    // the rear slot
        count++;
        return true;
    }
    bool deQueue() {
        if (isEmpty()) return false;
        front_ = (front_ + 1) % cap;             // just walk the front forward
        count--;
        return true;
    }
    int  Front()  { return isEmpty() ? -1 : data[front_]; }
    int  Rear()   { return isEmpty() ? -1 : data[(front_ + count - 1) % cap]; }
    bool isEmpty(){ return count == 0; }
    bool isFull() { return count == cap; }
};
```

WATCH OUT

This is asked because of one ambiguity: with `front` and `rear` alone, a **full** queue and an **empty** queue both satisfy `front == rear`. Keeping a `count` removes the ambiguity entirely and gives you `isFull` and `isEmpty` for free. The alternatives — always leave one slot empty, or keep a boolean flag — work too, but are fiddlier under pressure.

07 LC 150 · Evaluate Reverse Polish Notation

MEDIUM Pattern 2 — Expression evaluation

PROBLEM

Evaluate an arithmetic expression given in postfix (reverse Polish) notation. Division truncates toward zero.

APPROACH

The easiest algorithm in the topic: push operands, and on an operator pop two, apply, push the result. Postfix needs no precedence rules and no parentheses — the token order alone fixes the evaluation order, which is why compilers convert to it.

The final answer is the single value left on the stack.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<long long> st;
        for (const string& t : tokens) {
            if (t == "+" || t == "-" || t == "*" || t == "/") {
                long long b = st.top(); st.pop();      // RIGHT operand pops FIRST
                long long a = st.top(); st.pop();
                if      (t == "+") st.push(a + b);
                else if (t == "-") st.push(a - b);
                else if (t == "*") st.push(a * b);
                else               st.push(a / b);     // truncates toward zero
            } else {
                st.push(stoll(t));
            }
        }
        return (int)st.top();
    }
};
```

WATCH OUT

The first value you pop is the RIGHT operand. Get it backwards and `+` and `*` still pass every test while `-` and `/` silently fail. Pop `b` first, then `a`, and compute `a - b`. Check it on `["3","4","-"]`, which must give `-1`, not `1`. Use `long long` for the intermediate values — the constraints allow products that overflow `int`.

08 LC 682 · Baseball Game

EASY Pattern 10 — Stack simulation

PROBLEM

Process a list of operations on a record of scores: an integer adds a score, `"C"` cancels the previous score, `"D"` adds double the previous, `"+"` adds the sum of the previous two. Return the total.

APPROACH

A direct simulation. Use a `vector` rather than `std::stack`, because `"+"` needs the **two** most recent entries and `std::stack` only exposes one.

That is the practical lesson of this easy problem: when you need to look more than one element deep, a `vector` used as a stack is the right container.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int calPoints(vector<string>& ops) {
        vector<int> st;                     // a vector so we can read st[n-2]
        for (const string& op : ops) {
            if (op == "C") {
                st.pop_back();                          // cancel the last score
            } else if (op == "D") {
                st.push_back(st.back() * 2);
            } else if (op == "+") {
                st.push_back(st[st.size() - 1] + st[st.size() - 2]);
            } else {
                st.push_back(stoi(op));
            }
        }
        return accumulate(st.begin(), st.end(), 0);
    }
};
```

WATCH OUT

`std::stack` cannot be indexed or iterated, so `st[st.size()-2]` does not compile. Reach for `vector` with `push_back` / `back` / `pop_back` — the same interface, plus random access. This habit pays off again in every monotonic-stack problem where you need `st.top()` after a pop.

09 LC 1047 · Remove All Adjacent Duplicates In String

EASY Pattern 3 — String collapse

PROBLEM

Repeatedly remove two adjacent and equal letters until no such pair remains. Return the final string.

APPROACH

One pass, and the **output string itself is the stack**. For each character: if it equals the last character of the output, pop that character; otherwise push.

One pass suffices even though the problem says 'repeatedly', because a removal can only ever create a new adjacent pair at the position you just popped — which is exactly where the next comparison happens.

Complexity O(n) time · O(n) space for the output

C++ SOLUTION

```cpp
class Solution {
public:
    string removeDuplicates(string s) {
        string st;                          // the output string IS the stack
        for (char c : s) {
            if (!st.empty() && st.back() == c) st.pop_back();   // cancel the pair
            else st.push_back(c);
        }
        return st;
    }
};
```

WATCH OUT

Do not simulate the 'repeatedly' literally with nested loops — that is O(n²) and unnecessary. Also, `std::string` already has `push_back`, `back` and `pop_back`, so no separate container is needed at all. The variant LC 1209 removes runs of `k` equal characters; there you push `{char, count}` pairs instead of bare characters.

10 LC 844 · Backspace String Compare

EASY Pattern 3 — String collapse

PROBLEM

Two strings contain `#` meaning backspace. Decide whether they are equal after all backspaces are applied.

APPROACH

Build each string with a stack: push ordinary characters, pop on `#`. Then compare the two results.

The pop condition is simply 'the incoming character is `#`' — the same collapse shape as LC 1047 with a different trigger.

Complexity O(n + m) time · O(n + m) space. The O(1)-space follow-up walks both strings **backwards** with two pointers, skipping characters as backspaces are counted.

C++ SOLUTION

```cpp
class Solution {
    string build(const string& s) {
        string st;
        for (char c : s) {
            if (c == '#') { if (!st.empty()) st.pop_back(); }   // guard the empty case
            else st.push_back(c);
        }
        return st;
    }
public:
    bool backspaceCompare(string s, string t) {
        return build(s) == build(t);
    }
};
```

WATCH OUT

A `#` on an already-empty string is a no-op, not an error — `"#####a"` builds to `"a"`. Guard the `pop_back()` with an empty check or you get undefined behaviour on the very first test. If the interviewer asks for O(1) space, the backward two-pointer walk is the expected answer.

11 LC 496 · Next Greater Element I

EASY Pattern 5 — Monotonic stack

PROBLEM

`nums1` is a subset of `nums2`. For each element of `nums1`, find the first element to its right **in** `nums2` that is greater, or `-1`.

APPROACH

Run the standard next-greater pass over `nums2`, recording the answers in a hash map from value to next-greater-value. Then look each element of `nums1` up.

The stack holds values in **decreasing** order — those are the elements still waiting for something bigger. When a bigger value arrives, everything smaller on the stack has found its answer, so pop them all and record.

Complexity O(n + m) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        unordered_map<int, int> nxt;          // value -> its next greater value
        stack<int> st;                        // values, DECREASING bottom -> top
        for (int v : nums2) {
            while (!st.empty() && st.top() < v) { nxt[st.top()] = v; st.pop(); }
            st.push(v);
        }                                     // whatever is left has no next greater
        vector<int> res;
        for (int v : nums1) res.push_back(nxt.count(v) ? nxt[v] : -1);
        return res;
    }
};
```

WATCH OUT

Anything left on the stack when the loop ends never found a greater element — which is why the result is pre-filled with `-1` rather than assigned in a second pass. This is the base template for LC 503, 739, 901 and 1019; learn it as one unit and the rest are edits.

12 LC 739 · Daily Temperatures

MEDIUM Pattern 5 — Monotonic stack, distances

PROBLEM

For each day, how many days you must wait for a warmer temperature. `0` if it never gets warmer.

APPROACH

Exactly LC 496, with one change: the stack holds **indices**, and the answer recorded at pop time is `i - st.top()` — the *distance*, not the value.

This is the problem that proves the Part 2.2 rule. A value-based stack simply cannot produce a distance, because the index is not recoverable from the value.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& t) {
        int n = t.size();
        vector<int> res(n, 0);              // 0 = no warmer day ever comes
        stack<int> st;                      // INDICES, temperatures decreasing
        for (int i = 0; i < n; i++) {
            while (!st.empty() && t[st.top()] < t[i]) {
                res[st.top()] = i - st.top();          // the DISTANCE, not the value
                st.pop();
            }
            st.push(i);
        }
        return res;
    }
};
```

WATCH OUT

The `while` inside the `for` looks like O(n²) and interviewers will probe it. The answer: *each index is pushed exactly once and popped at most once, so the inner loop runs at most n times across the whole run.* Note also that ties do not pop — `t[st.top()] < t[i]` is strict, so equal temperatures keep waiting, which is what 'warmer' means.

13 LC 933 · Number of Recent Calls

EASY Pattern 8 — Queue as a time window

PROBLEM

Count the pings that happened in the inclusive range `[t - 3000, t]`, given that `t` increases with every call.

APPROACH

A queue of timestamps. Push the new one, then pop from the front while `q.front() < t - 3000`. The answer is `q.size()`.

This is the purest illustration of a queue as a **sliding window over time**: the front is always the oldest ping, and the oldest is always the first to fall out of range.

Complexity **Amortised O(1)** per `ping` · O(w) space, where w is the number of pings in a 3000 ms window

C++ SOLUTION

```cpp
class RecentCounter {
    queue<int> q;                           // ping times, oldest at the front
public:
    RecentCounter() {}

    int ping(int t) {
        q.push(t);
        while (q.front() < t - 3000) q.pop();   // drop everything now out of range
        return q.size();
    }
};
```

WATCH OUT

Because timestamps are strictly increasing, no element ever re-enters the window — so each timestamp is pushed once and popped once, and the loop is amortised O(1) despite looking like O(n). The same shape solves LC 346 (Moving Average) with a running sum added.

Tier 2 — The placement core

These are what actually get asked. 84, 42, 239 and 227 are the four most-asked stack and queue questions in on-campus drives — be able to write all four cold.

14 LC 503 · Next Greater Element II

MEDIUM Pattern 5 — Monotonic stack, circular

PROBLEM

Same as next-greater, but the array is **circular**: after the last element you may wrap around to the first.

APPROACH

The universal trick for circular arrays: run the loop `2n` **times** and index with `i % n`. Two laps guarantee every element sees every other element to its right, wrapping included.

Everything else is the LC 496 template unchanged.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& a) {
        int n = a.size();
        vector<int> res(n, -1);
        stack<int> st;                      // indices, values decreasing
        for (int i = 0; i < 2 * n; i++) {   // two laps around the circle
            int cur = a[i % n];
            while (!st.empty() && a[st.top()] < cur) { res[st.top()] = cur; st.pop(); }
            if (i < n) st.push(i);          // only push on the FIRST lap
        }
        return res;
    }
};
```

WATCH OUT

`if (i < n) st.push(i)` — only push during the **first** lap. Without that guard the second lap pushes duplicate indices, the stack never drains, and answers get overwritten. The second lap exists purely to *resolve* leftovers, not to add new ones.

15 LC 901 · Online Stock Span

MEDIUM Pattern 5 — Previous greater, streaming

PROBLEM

For each incoming price, return the number of consecutive days up to and including today whose price was less than or equal to today's.

APPROACH

This is *previous greater element*, phrased as a stream. The span is the distance back to the last strictly greater price.

Because it is online, the stack lives as a member variable between calls. Push `{price, span}` pairs and, when you pop a smaller price, **absorb its span into your own** — so you never need indices or a day counter at all.

Complexity **Amortised O(1)** per call · O(n) space

C++ SOLUTION

```cpp
class StockSpanner {
    // Each entry is {price, span}. The stack is decreasing in price, and the
    // span of a popped entry is absorbed into the entry that swallowed it.
    stack<pair<int, int>> st;
public:
    StockSpanner() {}

    int next(int price) {
        int span = 1;
        while (!st.empty() && st.top().first <= price) {
            span += st.top().second;         // inherit the days it already covered
            st.pop();
        }
        st.push({price, span});
        return span;
    }
};
```

WATCH OUT

The comparison is `<=`, not `<`: equal prices are part of the same span. And the span accumulation `span += st.top().second` is what makes this O(1) amortised — popping a run of prices in one step inherits all the days they covered. Recomputing by walking back would be O(n) per call.

16 LC 84 · Largest Rectangle in Histogram

HARD Pattern 6 — Monotonic stack, area ⭐

PROBLEM

Given bar heights of width 1, find the area of the largest axis-aligned rectangle that fits inside the histogram.

APPROACH

Every maximal rectangle is limited in height by some bar. For that bar, the rectangle extends left until a shorter bar and right until a shorter bar — so its width is `nextSmaller - prevSmaller - 1`.

One increasing stack computes both boundaries. When bar `i` pops a taller bar, `i` **is** its next smaller, and whatever is exposed underneath **is** its previous smaller. One pass, both bounds, O(n).

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int largestRectangleArea(vector<int>& h) {
        h.push_back(0);                     // sentinel: forces the stack to drain
        stack<int> st;                      // indices, heights INCREASING
        long long best = 0;
        for (int i = 0; i < (int)h.size(); i++) {
            while (!st.empty() && h[st.top()] > h[i]) {
                long long height = h[st.top()]; st.pop();
                // after the pop, st.top() is the previous SMALLER bar, which is
                // outside the rectangle -- hence the "- 1".
                int left = st.empty() ? -1 : st.top();
                best = max(best, height * (i - left - 1));
            }
            st.push(i);
        }
        h.pop_back();                       // leave the caller's vector as we found it
        return (int)best;
    }
};
```

WATCH OUT

Two details are the whole difficulty. **The `0` sentinel appended to the input** forces every remaining bar to be popped — without it, an increasing input like `[1,2,3]` returns 0. And the width is **`i - left - 1`**, not `i - st.top()`, because the exposed bar underneath is the *boundary* and lies outside the rectangle. Use `long long` for the product. This is the parent problem of LC 85.

17 LC 42 · Trapping Rain Water

HARD Pattern 6 — Monotonic stack, area ⭐

PROBLEM

Given an elevation map of bar heights, compute how much rain water is trapped between the bars.

APPROACH

The observation behind every solution: the water above column `i` is `min(maxLeft, maxRight) - height[i]`, floored at zero. Water is held by the **shorter** of the two walls.

Three valid solutions. **Prefix arrays** — precompute both maxima, O(n) space, impossible to get wrong. **Two pointers** — move whichever side has the smaller running maximum, since that side's answer is already decided; O(1) space, and the one to give first. **Monotonic stack** — fills water in horizontal *layers*: a popped bar is the floor of a puddle walled by the new bar on the right and the exposed stack top on the left.

> **Complexity** O(n) time. Space: O(n) for prefix arrays or the stack, **O(1)** for two pointers

C++ SOLUTION

```cpp
class Solution {
public:
    // Two pointers: O(n) time, O(1) space. Move whichever side has the smaller
    // running maximum, because that side's water level is already decided.
    int trap(vector<int>& a) {
        int l = 0, r = (int)a.size() - 1, lMax = 0, rMax = 0, water = 0;
        while (l < r) {
            if (a[l] < a[r]) {
                lMax = max(lMax, a[l]);
                water += lMax - a[l];
                l++;
            } else {
                rMax = max(rMax, a[r]);
                water += rMax - a[r];
                r--;
            }
        }
        return water;
    }

    // The monotonic-stack version: fills water in horizontal layers instead.
    int trapStack(vector<int>& a) {
        stack<int> st;                      // indices, heights decreasing
        int water = 0;
        for (int i = 0; i < (int)a.size(); i++) {
            while (!st.empty() && a[st.top()] < a[i]) {
                int floorIdx = st.top(); st.pop();
                if (st.empty()) break;                  // no left wall -> no puddle
                int width  = i - st.top() - 1;
                int height = min(a[st.top()], a[i]) - a[floorIdx];
                water += width * height;
            }
            st.push(i);
        }
        return water;
```

```
    }
};
```

WATCH OUT

In the stack version, `if (st.empty()) break;` after the pop is essential — with no left wall there is no puddle, and skipping the check reads a bogus boundary. Interviewers on this problem almost always ask for a second approach, so know at least two and be ready to say which is O(1) space and why.

18 LC 239 · Sliding Window Maximum

HARD Pattern 7 — Monotonic deque ⭐

PROBLEM

Given an array and a window size `k`, return the maximum of every window as it slides one position at a time.

APPROACH

A **deque of indices**, kept in decreasing order of value: it holds exactly the elements that could still be the maximum of some future window. Three steps per index, in this order:

1. **Expire** the front if `dq.front() <= i - k` — it has slid out of the window. 2. **Dominate** from the back while `a[dq.back()] <= a[i]` — a smaller element that arrived *earlier* can never win again, because `a[i]` outlives it and beats it. 3. **Push** `i`, then read `a[dq.front()]` as the window maximum once `i >= k - 1`.

Complexity O(n) time — each index enters and leaves the deque once · O(k) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& a, int k) {
        deque<int> dq;                      // INDICES, values decreasing
        vector<int> res;
        for (int i = 0; i < (int)a.size(); i++) {
            // 1. expire the front if it has slid out of the window
            if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
            // 2. anything smaller that arrived earlier can never win again
            while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
            // 3. this index is a candidate
            dq.push_back(i);
            if (i >= k - 1) res.push_back(a[dq.front()]);   // front is the maximum
        }
        return res;
    }
};
```

WATCH OUT

It must be a **deque**, not a stack: you remove from the back for domination and from the front for expiry, and no single-ended container can do both. Store **indices**, not values — expiry is an index comparison and a value-based deque cannot do it at all. For the sliding-window *minimum*, flip the domination test to `>=`.

19 LC 71 · Simplify Path

MEDIUM Pattern 3 — String collapse

PROBLEM

Convert an absolute Unix-style path into its canonical form: collapse `.`, resolve `..`, and squeeze repeated slashes.

APPROACH

Split on `/` and push each component onto a stack of directory names. `.` and the empty component (from `//`) are skipped; `..` pops one directory if there is one. Join what remains with `/`.

A `stringstream` with `getline(ss, part, '/')` does the splitting in two lines and handles the empty components for free.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    string simplifyPath(string path) {
        vector<string> st;                  // the canonical directory components
        stringstream ss(path);
        string part;
        while (getline(ss, part, '/')) {
            if (part.empty() || part == ".") continue;       // "//" and "/./"
            if (part == "..") { if (!st.empty()) st.pop_back(); }   // go up, if we can
            else st.push_back(part);
        }
        string res;
        for (const string& d : st) res += "/" + d;
        return res.empty() ? "/" : res;      // the root is a special case
    }
};
```

WATCH OUT

`..` at the root is a **no-op**, not an error — guard the pop with `if (!st.empty())`. And if the stack is empty at the end the answer is `"/"`, not the empty string. Both edge cases are in the sample tests precisely because everyone misses one of them.

20 LC 394 · Decode String

MEDIUM Pattern 4 — Nested context save & restore

PROBLEM

Decode a string of the form `k[encoded]`, where the bracketed text repeats `k` times and the encodings may nest — `3[a2[c]]` decodes to `accaccacc`.

APPROACH

On `[`, push the **context you are about to overwrite** — the repeat count and the text built so far — then start fresh. On `]`, pop them back and append the inner text `k` times.

That save-and-restore move is the general answer to any nested structure: brackets, calculators, directory paths, DFS. The stack is holding the outer levels that are not finished yet.

Complexity O(total output length) time · O(depth + output) space

C++ SOLUTION

```cpp
class Solution {
public:
    string decodeString(string s) {
        stack<int> counts;                  // repeat counts waiting for their ']'
        stack<string> parts;                // text built before each '['
        string cur;
        int k = 0;
        for (char c : s) {
            if (isdigit(c)) {
                k = k * 10 + (c - '0');     // counts can be multi-digit: 100[a]
            } else if (c == '[') {
                counts.push(k);             // save the context we are overwriting
                parts.push(cur);
                k = 0; cur.clear();
            } else if (c == ']') {
                string inner = cur;
                cur = parts.top(); parts.pop();          // restore the outer text
                int rep = counts.top(); counts.pop();
                while (rep--) cur += inner;
            } else {
                cur += c;
            }
        }
        return cur;
    }
};
```

WATCH OUT

Counts can be **multi-digit** — `100[a]` is legal — so accumulate with `k = k*10 + (c - '0')` rather than reading one character. Reset `k` and `cur` after pushing, or the inner level inherits the outer level's state. Two parallel stacks (counts, texts) or one stack of pairs both work.

21 LC 227 · Basic Calculator II

MEDIUM Pattern 2 — Expression evaluation ⭐

PROBLEM

Evaluate an infix expression containing non-negative integers and the operators `+ - * /`, with no parentheses. Integer division truncates toward zero.

APPROACH

Additive terms can be deferred; multiplicative ones cannot. So push each number as a **signed term** — `+` pushes it, `-` pushes its negative — and when the pending operator was `*` or `/`, fold it into the top of the stack immediately. The answer is the sum of the whole stack.

The design hinges on `op` holding the operator that came *before* the number just read, not after it. That one-step delay is what makes precedence come out right without a second stack.

Complexity O(n) time · O(n) space, reducible to O(1) by keeping only the last term instead of a stack

C++ SOLUTION

```cpp
class Solution {
public:
    int calculate(string s) {
        stack<long long> st;
        long long num = 0;
        char op = '+';                      // the operator BEFORE the current number
        for (int i = 0; i < (int)s.size(); i++) {
            char c = s[i];
            if (isdigit(c)) num = num * 10 + (c - '0');
            // flush on any operator, and on the last character whatever it is
            if ((!isdigit(c) && c != ' ') || i + 1 == (int)s.size()) {
                if      (op == '+') st.push(num);
                else if (op == '-') st.push(-num);       // subtraction = a negative term
                else if (op == '*') { long long t = st.top(); st.pop(); st.push(t * num); }
                else                { long long t = st.top(); st.pop(); st.push(t / num); }
                op = c; num = 0;
            }
        }
        long long sum = 0;
        while (!st.empty()) { sum += st.top(); st.pop(); }
        return (int)sum;
    }
};
```

WATCH OUT

Flush the final number. The last digit run has no operator after it to trigger the push, so the condition needs `|| i + 1 == s.size()`. Without it the last term of every expression is silently dropped. Also skip spaces — they are legal anywhere, including in the middle of an expression.

22 LC 946 · Validate Stack Sequences

MEDIUM Pattern 10 — Stack simulation

PROBLEM

Given a `pushed` sequence and a `popped` sequence of distinct values, decide whether some interleaving of pushes and pops produces exactly that pop order.

APPROACH

Simulate it greedily. Push each value in order; after every push, pop for as long as the stack top equals the next value expected in `popped`.

Greedy is provably correct here: if the top **is** the next required value, popping now is never worse, because it will have to come off before anything pushed after it anyway. The sequence is valid exactly when the stack ends empty.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool validateStackSequences(vector<int>& pushed, vector<int>& popped) {
        stack<int> st;
        size_t j = 0;                       // how much of popped we have matched
        for (int v : pushed) {
            st.push(v);
            // pop greedily for as long as the top is what must come out next
            while (!st.empty() && j < popped.size() && st.top() == popped[j]) {
                st.pop(); j++;
            }
        }
        return st.empty();                  // everything came out in the right order
    }
};
```

WATCH OUT

Guard the inner loop with **both** `!st.empty()` and `j < popped.size()`, or you index past the end on the last pop. And the final answer is `st.empty()`, not `j == popped.size()` — although with valid input the two coincide, only the first is safe.

23 LC 402 · Remove K Digits

MEDIUM Pattern 5 — Greedy monotonic stack

PROBLEM

Remove exactly `k` digits from a numeric string so the remaining number is the smallest possible.

APPROACH

Greedy: a bigger digit standing in front of a smaller one always makes the number larger, so remove it. That is precisely an **increasing** monotonic stack — pop while the top exceeds the incoming digit and removals remain.

If the string is already non-decreasing nothing pops during the loop, so any leftover removals come off the **end**, where the least significant digits are.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    string removeKdigits(string num, int k) {
        string st;                          // an INCREASING stack of digits
        for (char c : num) {
            // a bigger digit in front of a smaller one is always worth removing
            while (k > 0 && !st.empty() && st.back() > c) { st.pop_back(); k--; }
            st.push_back(c);
        }
        while (k-- > 0 && !st.empty()) st.pop_back();   // digits already increasing
        int i = 0;
        while (i < (int)st.size() && st[i] == '0') i++; // strip leading zeros
        string res = st.substr(i);
        return res.empty() ? "0" : res;
    }
};
```

WATCH OUT

Three finishing steps, and every one of them is a test case. **Spend leftover** `k` by trimming from the back (`"112", k=1` → `"11"`). **Strip leading zeros** (`"10200", k=1` → `"200"`, not `"0200"`). **Return** `"0"` **if nothing is left** (`"10", k=2`). LC 1673 and LC 316 are the same greedy with different pop conditions.

24 LC 856 · Score of Parentheses

MEDIUM Pattern 4 — Nested context

PROBLEM

Score a balanced string where `()` scores 1, `AB` scores `A + B`, and `(A)` scores `2 * A`.

APPROACH

Keep a stack of **per-level scores**, seeded with a 0 for the outermost level. `(` opens a new level by pushing 0. `)` closes the current level: pop its score, convert it — `0` means the level was empty so it scores 1, otherwise it doubles — and add the result into the level below.

The stack holds one running score per unfinished nesting level, which is the Pattern 4 shape with a number instead of a string as the saved context.

Complexity O(n) time · O(depth) space

C++ SOLUTION

```cpp
class Solution {
public:
    int scoreOfParentheses(string s) {
        stack<int> st;
        st.push(0);                          // the score of the current (outermost) level
        for (char c : s) {
            if (c == '(') {
                st.push(0);                  // open a new, empty level
            } else {
                int inner = st.top(); st.pop();
                int add = inner ? 2 * inner : 1;     // "()" scores 1, "(A)" scores 2A
                st.top() += add;                     // fold into the enclosing level
            }
        }
        return st.top();
    }
};
```

WATCH OUT

The `inner ? 2 * inner : 1` line encodes both rules at once, and the `AB` rule falls out for free because sibling levels accumulate into the same parent slot. Seeding the stack with one 0 before the loop removes the 'there is no enclosing level' special case — the same sentinel idea as LC 32 and LC 84.

25 LC 1249 · Minimum Remove to Make Valid Parentheses

MEDIUM Pattern 1 — Indices of unmatched brackets

PROBLEM

Remove the minimum number of brackets from a string of letters and parentheses so the result is valid. Any valid answer is accepted.

APPROACH

Two passes. First, scan once with a stack of the **indices** of unmatched `(`; every `)` that arrives with an empty stack is an orphan, so mark it for deletion. Whatever indices remain on the stack at the end are unmatched openers, so mark those too.

Second, rebuild the string skipping every marked index. Marking indices rather than erasing as you go keeps the pass O(n) and avoids invalidating positions.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    string minRemoveToMakeValid(string s) {
        stack<int> open;                    // INDICES of '(' still unmatched
        vector<bool> drop(s.size(), false);
        for (int i = 0; i < (int)s.size(); i++) {
            if (s[i] == '(') open.push(i);
            else if (s[i] == ')') {
                if (open.empty()) drop[i] = true;    // an orphan ')'
                else open.pop();
            }
        }
        while (!open.empty()) { drop[open.top()] = true; open.pop(); }  // orphan '('
        string res;
        for (int i = 0; i < (int)s.size(); i++) if (!drop[i]) res += s[i];
        return res;
    }
};
```

WATCH OUT

This is why you push **indices** rather than characters — you cannot delete a character you cannot locate. Do not try to erase in place during the scan: each `erase` is O(n) and every later index shifts. Letters are untouched and must be preserved in order.

26 LC 32 · Longest Valid Parentheses

HARD Pattern 1 — the −1 sentinel

PROBLEM

Find the length of the longest **contiguous** valid parentheses substring.

APPROACH

Keep a stack of indices whose bottom is always the index of the **last unmatched character** — the base of the current valid run. Seed it with `-1`.

On `(`, push the index. On `)`, pop: if the stack is now empty, this `)` is itself unmatched, so it becomes the new base and gets pushed. Otherwise the current run is valid all the way back to the base, and its length is `i - st.top()`.

Complexity O(n) time · O(n) space. A two-pass counter scan (left to right, then right to left) solves it in O(1) space

C++ SOLUTION

```cpp
class Solution {
public:
    int longestValidParentheses(string s) {
        stack<int> st;
        st.push(-1);                        // sentinel: the base of the current run
        int best = 0;
        for (int i = 0; i < (int)s.size(); i++) {
            if (s[i] == '(') {
                st.push(i);
            } else {
                st.pop();                   // try to match it
                if (st.empty()) st.push(i);  // no match: i becomes the new base
                else best = max(best, i - st.top());   // length from the base to here
            }
        }
        return best;
    }
};
```

WATCH OUT

The `-1` seed is what makes a run starting at index 0 measure correctly without a special case — exactly the same sentinel idea as the trailing `0` in LC 84. The measurement is `i - st.top()` **after** the pop, using what is now exposed; measuring against the popped value gives the length of one pair, not the run.

27 LC 735 · Asteroid Collision

MEDIUM Pattern 10 — Stack simulation

PROBLEM

Asteroids move left (negative) or right (positive) at the same speed. On collision the smaller explodes; equal sizes destroy both. Return the state after all collisions.

APPROACH

A stack of survivors. A collision is possible **only** when the stack top moves right (`> 0`) and the incoming asteroid moves left (`< 0`) — every other pairing never meets.

While that condition holds, compare sizes: pop the stack one if it is smaller, pop both and kill the incoming one if equal, or kill the incoming one if the stack one is bigger.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        vector<int> st;                       // the survivors so far
        for (int a : asteroids) {
            bool alive = true;
            // a collision happens only when a right-mover is on the stack and
            // the incoming asteroid moves left
            while (alive && a < 0 && !st.empty() && st.back() > 0) {
                if (st.back() < -a)      st.pop_back();      // the stack one explodes
                else if (st.back() == -a){ st.pop_back(); alive = false; }  // both do
                else                      alive = false;     // the incoming one does
            }
            if (alive) st.push_back(a);
        }
        return st;
    }
};
```

WATCH OUT

Use an explicit `alive` flag rather than trying to `break` out of nested logic — an incoming asteroid may destroy several stack asteroids in a row before surviving, or may die partway through. Getting the three-way size comparison right (`<`, `==`, `>`) is the whole problem; the `==` case must pop **and** kill.

28 LC 994 · Rotting Oranges

MEDIUM Pattern 9 — Multi-source BFS

PROBLEM

In a grid of empty cells, fresh oranges and rotten oranges, a rotten orange rots its four neighbours each minute. Return the minutes until none are fresh, or `-1` if impossible.

APPROACH

Multi-source BFS: push *every* initially rotten orange into the queue before starting, so all sources spread simultaneously. Count fresh oranges up front and decrement as each one rots.

Process the queue **one level at a time** — `int sz = q.size()` frozen before the inner loop — and increment the minute counter after each full level. If any fresh oranges remain when the queue empties, return `-1`.

Complexity O(m·n) time · O(m·n) space for the queue

C++ SOLUTION

```cpp
class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        int m = grid.size(), n = grid[0].size(), fresh = 0;
        queue<pair<int, int>> q;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 2) q.push({i, j});   // MULTI-source: every rotten one
                else if (grid[i][j] == 1) fresh++;
            }
        if (fresh == 0) return 0;                      // nothing to rot
        int minutes = 0;
        int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
        while (!q.empty() && fresh > 0) {
            int sz = q.size();                         // FREEZE the level width
            for (int s = 0; s < sz; s++) {
                auto [r, c] = q.front(); q.pop();
                for (int d = 0; d < 4; d++) {
                    int nr = r + dr[d], nc = c + dc[d];
                    if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
                    if (grid[nr][nc] != 1) continue;
                    grid[nr][nc] = 2;                  // mark on push, not on pop
                    fresh--;
                    q.push({nr, nc});
                }
            }
            minutes++;                                 // one full level = one minute
        }
        return fresh == 0 ? minutes : -1;
    }
};
```

WATCH OUT

Mark a cell as rotten **when you push it**, not when you pop it, or the same cell enters the queue several times. Freeze `q.size()` before the inner loop, or the oranges you just pushed join the current minute and the count is wrong — that is bug #7. Return `0`, not `1`, when there were no fresh oranges to begin with.

Tier 3 — Harder, but they do appear

Extensions of the same two ideas: the monotonic stack applied to areas and contributions, and the monotonic deque applied to prefix sums and DP. Attempt these once Tier 2 is comfortable.

29 LC 85 · Maximal Rectangle

HARD Pattern 6 — Histogram per row

PROBLEM

Given a binary matrix of `'0'` and `'1'` characters, find the area of the largest rectangle containing only ones.

APPROACH

Reduce it to LC 84. Walk the rows top to bottom maintaining a **running histogram**: `heights[c]` is the number of consecutive `'1'`s ending at the current row in column `c`, reset to 0 wherever a `'0'` appears.

After updating the histogram for a row, run Largest Rectangle in Histogram on it. The best over all rows is the answer.

Complexity O(m·n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
    // exactly LC 84, reused unchanged
    int largestRectangleArea(vector<int>& h) {
        h.push_back(0);
        stack<int> st;
        long long best = 0;
        for (int i = 0; i < (int)h.size(); i++) {
            while (!st.empty() && h[st.top()] > h[i]) {
                long long height = h[st.top()]; st.pop();
                int left = st.empty() ? -1 : st.top();
                best = max(best, height * (i - left - 1));
            }
            st.push(i);
        }
        h.pop_back();
        return (int)best;
    }
public:
    int maximalRectangle(vector<vector<char>>& matrix) {
        if (matrix.empty() || matrix[0].empty()) return 0;
        int n = matrix[0].size(), best = 0;
        vector<int> heights(n, 0);
        for (const auto& row : matrix) {
            for (int c = 0; c < n; c++)
                heights[c] = (row[c] == '1') ? heights[c] + 1 : 0;  // running histogram
            best = max(best, largestRectangleArea(heights));
        }
```

```
        return best;
    }
};
```

WATCH OUT

Recognising the reduction **is** the problem — the rest is code you already have. The matrix holds `char` values `'0'`/`'1'`, not integers, so compare against `'1'` and not `1`. And the histogram must be updated in place across rows, never rebuilt from scratch, or you lose the vertical accumulation that makes it work.

30 LC 224 · Basic Calculator

HARD Pattern 4 — Nested context

PROBLEM

Evaluate an infix expression with non-negative integers, `+`, `-`, unary minus and **parentheses**. No multiplication or division.

APPROACH

Different technique from LC 227, because parentheses nest and `*`/`/` do not appear. Keep a running `result` and a running `sign`. On `(`, push the **context** — the result so far and the sign standing in front of the bracket — and reset both. On `)`, finish the sub-expression, multiply by the saved sign and add the saved result back.

So the stack alternates `result, sign, result, sign, …`, one pair per unfinished nesting level.

Complexity O(n) time · O(depth) space

C++ SOLUTION

```cpp
class Solution {
public:
    int calculate(string s) {
        stack<int> st;                // alternating: result, sign, result, sign, ...
        int result = 0, sign = 1, num = 0;
        for (char c : s) {
            if (isdigit(c)) {
                num = num * 10 + (c - '0');
            } else if (c == '+') {
                result += sign * num; num = 0; sign =  1;
            } else if (c == '-') {
                result += sign * num; num = 0; sign = -1;
            } else if (c == '(') {
                st.push(result);      // save the context we are about to overwrite
                st.push(sign);
                result = 0; sign = 1;
            } else if (c == ')') {
                result += sign * num; num = 0;
                result *= st.top(); st.pop();       // the sign in front of the '('
                result += st.top(); st.pop();       // the result before the '('
                sign = 1;
            }
        }
        return result + sign * num;    // flush the trailing number
    }
};
```

WATCH OUT

The two pops at `)` must come in the reverse order of the two pushes at `(` — sign was pushed last, so it pops first. Flush the trailing number with `result + sign * num` after the loop; without it the last term vanishes. Unary minus works for free: `sign` starts at `+1` and a `-` simply flips it.

31 LC 907 · Sum of Subarray Minimums

MEDIUM Pattern 6 — Contribution counting

PROBLEM

Sum `min(subarray)` over **every** contiguous subarray, modulo 1e9+7.

APPROACH

Do not enumerate subarrays. Instead ask, for each element, **how many subarrays it is the minimum of**. If `a[i]` extends left to `prevSmaller[i]` and right to `nextSmaller[i]`, that count is `(i - prev) * (next - i)`, and the answer is the sum of `a[i]` times that count.

Both boundary arrays come out of the single eight-line monotonic pass from Part 4.2.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int sumSubarrayMins(vector<int>& a) {
        const long long MOD = 1000000007LL;
        int n = a.size();
        vector<int> prev(n, -1), next(n, n);
        stack<int> st;
        // STRICT on the left, NON-STRICT on the right: among equal values exactly
        // one is treated as the owner, so no subarray is counted twice.
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.top()] >= a[i]) { next[st.top()] = i; st.pop(); }
            prev[i] = st.empty() ? -1 : st.top();
            st.push(i);
        }
        long long total = 0;
        for (int i = 0; i < n; i++) {
            long long left  = i - prev[i];      // choices of start
            long long right = next[i] - i;      // choices of end
            total = (total + (long long)a[i] % MOD * (left * right % MOD)) % MOD;
        }
        return (int)total;
    }
};
```

WATCH OUT

Duplicates are the entire difficulty. Use a strict comparison on one side and non-strict on the other — here `>=` when popping (so the *right* boundary stops at the first value less than **or equal**) and strict on the left. Symmetric comparisons double-count every subarray whose minimum appears twice; test on `[2,2,2]`, which must give 12. Take the modulus during accumulation, and keep the products in `long long`.

32 LC 456 · 132 Pattern

MEDIUM Pattern 5 — Right-to-left monotonic stack

PROBLEM

Decide whether there exist indices `i < j < k` with `a[i] < a[k] < a[j]` — a value, then a bigger one, then one in between.

APPROACH

Scan **right to left** with a decreasing stack. Maintain `third`: the largest value you have popped so far, which by construction has a strictly greater value somewhere to its right — so it is a valid '2' with a '3' behind it.

Then the moment any element is smaller than `third`, it is the '1' and the pattern exists. Elements pop only when a bigger value arrives, which is exactly what makes the popped one a valid '2'.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool find132pattern(vector<int>& a) {
        // Scan RIGHT to LEFT. "third" is the largest value we have already
        // popped, so it is guaranteed to have a bigger value to its right.
        stack<int> st;                      // decreasing bottom -> top
        int third = INT_MIN;
        for (int i = (int)a.size() - 1; i >= 0; i--) {
            if (a[i] < third) return true;    // a[i] < third < the popper: 1-3-2 found
            while (!st.empty() && st.top() < a[i]) { third = st.top(); st.pop(); }
            st.push(a[i]);
        }
        return false;
    }
};
```

WATCH OUT

The direction is the trick: left to right you would have to know the future. Check `a[i] < third` **before** the popping loop for index `i`, not after — checking afterwards lets the current element update `third` and then compare against itself. `third` starts at `INT_MIN` so the first comparison always fails, as it should.

33 LC 316 · Remove Duplicate Letters

MEDIUM Pattern 5 — Greedy monotonic + lookahead

PROBLEM

Remove duplicate letters so that every letter appears exactly once, and the result is the lexicographically smallest such string.

APPROACH

An increasing stack, like LC 402, but with two extra conditions. Precompute each letter's **last** index. Then for each character: skip it if it is already in the result, and pop bigger letters off the top only if they are **guaranteed to reappear later** (`last[top] > i`).

A `used[]` array keeps membership O(1) and must be cleared whenever a letter is popped.

Complexity O(n) time · O(1) extra space — at most 26 letters on the stack

C++ SOLUTION

```cpp
class Solution {
public:
    string removeDuplicateLetters(string s) {
        int last[26] = {0};
        for (int i = 0; i < (int)s.size(); i++) last[s[i] - 'a'] = i;  // final position
        bool used[26] = {false};
        string st;                              // an INCREASING stack of letters
        for (int i = 0; i < (int)s.size(); i++) {
            char c = s[i];
            if (used[c - 'a']) continue;      // already placed, and placement is final
            // drop a bigger letter only if it is guaranteed to reappear later
            while (!st.empty() && st.back() > c && last[st.back() - 'a'] > i) {
                used[st.back() - 'a'] = false;
                st.pop_back();
            }
            st.push_back(c);
            used[c - 'a'] = true;
        }
        return st;
    }
};
```

WATCH OUT

Both extra conditions are load-bearing. Without the lookahead you pop a letter that never comes back and the output is missing a character; without `used[]` you push duplicates. And when you pop, you must set `used[popped] = false` or that letter can never be re-added. LC 1081 is the identical problem under a different name.

34 LC 1019 · Next Greater Node In Linked List

MEDIUM Pattern 5 — Monotonic stack on a list

PROBLEM

For each node of a linked list, find the value of the first node after it that is strictly greater, or `0`.

APPROACH

Copy the list into a `vector` in one walk, then run the standard next-greater pass. Do not try to run the monotonic stack over the pointers directly — you would need indices anyway, and the extra O(n) space is already implied by the output array.

The pass itself is LC 496 verbatim, with `0` instead of `-1` as the 'none' value.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> nextLargerNodes(ListNode* head) {
        vector<int> a;                          // copy to a vector first -- do not
        for (ListNode* p = head; p; p = p->next) a.push_back(p->val);  // fight pointers
        vector<int> res(a.size(), 0);           // 0 = no greater node ahead
        stack<int> st;                          // indices, values decreasing
        for (int i = 0; i < (int)a.size(); i++) {
            while (!st.empty() && a[st.top()] < a[i]) { res[st.top()] = a[i]; st.pop(); }
            st.push(i);
        }
        return res;
    }
};
```

WATCH OUT

The default is `0`, not `-1` — values are positive, so `0` is the sentinel this problem chose. Converting the list to an array first is the *intended* solution, not a shortcut: knowing when to convert a data structure instead of fighting it is a real interview signal.

35 LC 636 · Exclusive Time of Functions

MEDIUM Pattern 4 — Nested context

PROBLEM

Given logs of `start`/`end` events for single-threaded, possibly recursive function calls, compute the exclusive running time of each function.

APPROACH

Simulate the call stack literally. Keep the ids of running functions on a stack and a `prev` timestamp marking how far time has already been accounted for.

On a `start`: credit the currently running function with `ts - prev`, push the new id, set `prev = ts`. On an `end`: credit the top with `ts - prev + 1`, pop, set `prev = ts + 1`.

Complexity O(n) time · O(depth) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> exclusiveTime(int n, vector<string>& logs) {
        vector<int> res(n, 0);
        stack<int> st;                      // function ids currently on the call stack
        int prev = 0;                       // the timestamp we last accounted for
        for (const string& log : logs) {
            int p1 = log.find(':'), p2 = log.rfind(':');
            int id       = stoi(log.substr(0, p1));
            string type  = log.substr(p1 + 1, p2 - p1 - 1);
            int ts       = stoi(log.substr(p2 + 1));
            if (type == "start") {
                if (!st.empty()) res[st.top()] += ts - prev;   // credit the caller
                st.push(id);
                prev = ts;
            } else {
                res[st.top()] += ts - prev + 1;   // "end" is INCLUSIVE of its own unit
                st.pop();
                prev = ts + 1;
            }
        }
        return res;
    }
};
```

WATCH OUT

The `+1` on `end`, and only on `end`. Timestamps are inclusive units of time, so a function that starts and ends at the same timestamp still ran for one unit. Correspondingly `prev` advances to `ts + 1` after an end but only to `ts` after a start. Getting either half wrong shifts every subsequent total. Recursion is handled for free — the same id can appear on the stack more than once.

36 LC 895 · Maximum Frequency Stack

HARD Pattern 11 — A stack per frequency

PROBLEM

Design a stack where `pop` removes the **most frequent** element, breaking ties by which was pushed most recently.

APPROACH

Keep a frequency map, and a **separate stack for every frequency level**. `push(x)` increments `freq[x]` to `f` and pushes `x` onto `group[f]`. `pop()` takes from `group[maxFreq]` and decrements `maxFreq` when that group empties.

The tie-break comes free: within one frequency level the values are already in push order, so the top is the most recent. No heap and no comparator are needed.

Complexity O(1) for both operations · O(n) space

C++ SOLUTION

```cpp
class FreqStack {
    unordered_map<int, int> freq;              // value -> how many copies are in
    unordered_map<int, vector<int>> group;     // frequency level -> stack of values
    int maxFreq = 0;
public:
    FreqStack() {}

    void push(int val) {
        int f = ++freq[val];
        maxFreq = max(maxFreq, f);
        group[f].push_back(val);               // one stack PER frequency level
    }
    int pop() {
        int val = group[maxFreq].back();       // most recent among the most frequent
        group[maxFreq].pop_back();
        freq[val]--;
        if (group[maxFreq].empty()) maxFreq--;
        return val;
    }
};
```

WATCH OUT

An element appears in **several** groups at once — pushing `5` three times leaves a copy in `group[1]`, `group[2]` and `group[3]`, which is exactly what makes the structure work when its frequency drops back down. `maxFreq` decreases by exactly one when a group empties, never more, because the levels below are always non-empty. Indexing containers by the quantity you are maximising is a reusable design idea.

37 LC 862 · Shortest Subarray with Sum at Least K

HARD Pattern 7 — Monotonic deque on prefix sums

PROBLEM

Find the length of the shortest **non-empty** contiguous subarray whose sum is at least `k`. Values may be negative.

APPROACH

Build prefix sums `p`, so a subarray sum is `p[i] - p[j]` with `j < i`. Keep a deque of indices into `p` with **increasing** values, and for each `i` do two things.

From the front: while `p[i] - p[dq.front()] >= k`, record `i - dq.front()` and pop it — that front can never give a shorter answer later, so it is done. **From the back:** while `p[dq.back()] >= p[i]`, pop — a prefix that is both larger *and* further left is dominated and can never win.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int shortestSubarray(vector<int>& a, int k) {
        int n = a.size();
        vector<long long> p(n + 1, 0);                      // prefix sums
        for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
        deque<int> dq;                          // indices into p, values INCREASING
        int best = n + 1;
        for (int i = 0; i <= n; i++) {
            // any front that already satisfies the sum gives a candidate, and it
            // can never be beaten by a later i, so pop it
            while (!dq.empty() && p[i] - p[dq.front()] >= k) {
                best = min(best, i - dq.front());
                dq.pop_front();
            }
            // a prefix that is >= the incoming one is useless: it is both larger
            // and further left, so it can never win
            while (!dq.empty() && p[dq.back()] >= p[i]) dq.pop_back();
            dq.push_back(i);
        }
        return best == n + 1 ? -1 : best;
    }
};
```

WATCH OUT

Negative numbers are what make this hard: the sliding-window technique that works for all-positive arrays fails, because the sum is no longer monotone in the window size. Iterate to `i <= n` inclusive, since `p` has `n + 1` entries. Use `long long` for the prefix sums.

38 LC 1425 · Constrained Subsequence Sum

HARD Pattern 7 — Monotonic deque + DP

PROBLEM

Choose a non-empty subsequence maximising its sum, subject to consecutive chosen indices differing by at most `k`. Return that maximum.

APPROACH

The DP is one line: `dp[i] = a[i] + max(0, max(dp[j]) for i-k <= j < i)`, where the `max(0, ...)` means you may start a fresh subsequence at `i`.

The inner maximum is a **sliding window maximum over `dp`**, so the LC 239 deque computes it in O(1) amortised per step instead of O(k). Expire the front when it falls out of the `k`-window, dominate from the back, and read the front.

Complexity O(n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int constrainedSubsetSum(vector<int>& a, int k) {
        int n = a.size();
        vector<int> dp(n);                    // dp[i] = best sum of a subsequence ending at i
        deque<int> dq;                        // indices, dp values DECREASING
        int best = INT_MIN;
        for (int i = 0; i < n; i++) {
            if (!dq.empty() && dq.front() < i - k) dq.pop_front();   // expire
            dp[i] = a[i] + (dq.empty() ? 0 : max(0, dp[dq.front()])); // best in the window
            best = max(best, dp[i]);
            while (!dq.empty() && dp[dq.back()] <= dp[i]) dq.pop_back();  // dominate
            dq.push_back(i);
        }
        return best;
    }
};
```

WATCH OUT

This is the capstone: a monotonic deque used as the *accelerator* inside a DP, not as the answer itself. The deque orders by `dp` values, **not** by the input values. The `max(0, ...)` is what allows a fresh start; drop it and negative-only arrays return the wrong answer. The final answer is `max(dp)`, not `dp[n-1]`.